SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-026: Forgot Password with Gmail OTP Verification and Password Reset

University College of Jaffna

Department of Interdisciplinary Studies

Information and Communication Technology

SCM (K72C001M12)

NVQ Level 6 – Semester

Field	Details
Change Request ID	CR-026
Change Request Title	Forgot Password with Gmail OTP Verification and Password Reset
Student Name	S. Hindusha
Student ID	JFICT24_26
Date	14 October 2026
Priority	High
Change Type	Frontend Enhancement and Backend Functionality
Implementation Status	Implemented – Pending Final Verification

1. CHANGE REQUEST SUMMARY

1.1 Business Requirement

The Student Management System required a password recovery facility to help registered users regain access to their accounts when they forget their passwords.

Previously, users could log in using their registered credentials, but the system did not provide a complete automated password recovery process. Therefore, a Forgot Password feature was required to allow users to verify their email addresses and create new passwords without depending on manual administrator assistance.

The new feature allows users to enter their registered email addresses and receive a six-digit One-Time Password (OTP) through Gmail. After entering the OTP, users can proceed to the Reset Password page, where they can enter a new password and confirm it.

The system validates the submitted information before updating the password. The new password is hashed using bcryptjs before being stored in the database.

The main objective of this change is to improve account accessibility, security, and user experience while maintaining the existing login and user-management functionality.

1.2 Existing Behaviour

Before implementing this change:

- Users could log in using their registered credentials.
- The existing authentication system verified passwords.
- Passwords were stored as hashes in the database.
- A complete automated email OTP password recovery workflow was not available.
- Users could not verify their identity through a dedicated OTP verification process.
- Users required a convenient way to create a new password.
- Password confirmation and additional validation were required.
- Appropriate recovery success and error messages were needed.

1.3 Requested Change

The requested change was to introduce a Forgot Password and Reset Password facility using Gmail OTP verification.

The main requirements were:

- Provide a Forgot Password page.
- Allow users to enter their registered email addresses.
- Generate a six-digit OTP.
- Send the OTP to the user's email address through Gmail.
- Provide an OTP verification interface.
- Reject incorrect and expired OTPs.
- Restrict repeated OTP verification attempts.
- Allow users to enter a new password after successful OTP verification.
- Provide a Confirm New Password field.
- Validate the password and confirmation fields.

- Enforce password-strength requirements on the backend.
- Hash the new password using bcryptjs.
- Update the existing password record in the database.
- Display appropriate success and error messages.
- Allow users to return to the Login page after a successful password reset.
- Preserve existing login, student dashboard, and administrator functionality.

2. IMPLEMENTATION

2.1 Implementation Summary

CR-026 was implemented by extending the existing Student Management System frontend and backend.

On the frontend, the password recovery interface was developed using React. The Forgot Password page allows users to submit their registered email addresses and request an OTP. The OTP verification page allows users to enter the six-digit code received through Gmail.

After successful OTP verification, the application navigates to the Reset Password page. The user can enter a new password and confirm it before submitting the request.

The authentication service was updated to communicate with the backend password recovery endpoints.

On the backend, the password recovery controller handles OTP generation, email delivery, OTP verification, password validation, and password updating.

Nodemailer is used for Gmail email delivery, while bcryptjs is used to hash the new password before it is stored in the database.

The implementation is intended to preserve the existing authentication and user-management functions.

2.2 Affected Files

The following files are relevant to the implementation. The final list should be checked against the actual project files.

No.	File Name	Purpose
1	frontend/src/pages/ForgotPasswordPage.jsx	Allows users to request an OTP using their registered email address.
2	frontend/src/pages/VerifyOTPPage.jsx	Provides the OTP verification interface.
3	frontend/src/pages/ResetPasswordPage.jsx	Allows users to enter and confirm a new password.
4	frontend/src/services/authService.js	Sends password recovery requests to the backend.
5	frontend/src/routes/AppRoutes.jsx	Registers the password recovery routes.
6	frontend/src/styles/passwordRecovery.css	Styles the password recovery pages.
7	backend/src/controllers/passwordResetController.js	Handles OTP requests, verification, and password resets.
8	backend/src/routes/authRoutes.js	Registers the password recovery API endpoints.
9	backend/src/models/passwordResetModel.js	Performs database operations related to OTP verification and password updates.
10	backend/src/services/smtpService.js	Sends OTP emails through Gmail.
11	backend/.env	Stores private email configuration and related settings.
12	MySQL database tables	Store OTP records and existing user password information.

2.3 Technical Changes

2.3.1 Forgot Password Page

The Forgot Password page was introduced to allow users to initiate password recovery.

The page provides an email input field, a Send OTP button, loading indicators, success and error messages, and a link to return to the Login page.

When a user submits an email address, the frontend sends a request to the backend. The backend validates the request and initiates the OTP delivery process.

This feature provides a convenient way for users to begin recovering their accounts.

2.3.2 OTP Verification Page

The OTP verification page allows users to enter the six-digit OTP received through Gmail.

The input field accepts numeric characters and limits the input to six digits. The frontend checks the OTP format before submitting the verification request.

The backend checks whether the OTP is valid and whether it has expired. Incorrect or expired OTPs must be rejected.

After successful verification, the application navigates to the Reset Password page using the email address and OTP information.

The password reset endpoint must independently verify the OTP before changing the password.

2.3.3 Reset Password Page

The Reset Password page allows users to create a new password after completing OTP verification.

The form contains the following fields:

- New Password
- Confirm New Password

The page also provides a Reset Password button and displays validation, success, and error messages.

Before submitting the request, the frontend checks that both fields are completed, the password meets the configured requirements, and the confirmation matches the new password.

The email address, OTP, new password, and confirmation password are sent to the backend for validation.

After a successful password update, the system displays a confirmation message and allows the user to return to the Login page.

2.3.4 Authentication Service Modification

The authService.js file was updated to provide the functions required for password recovery.

The main operations are:

Request Password OTP

Sends the registered email address to the backend to request an OTP.

Verify Password OTP

Submits the email address and OTP to the backend for verification.

Reset Password

Submits the email address, OTP, new password, and confirmation password to the backend.

Separating API communication from the React components improves code organisation and makes the authentication functions easier to maintain.

2.3.5 Route Modification

The application route configuration was updated to include the password recovery pages.

The following routes are used:

```
<Route
   path="/forgot-password"
   element={<ForgotPasswordPage />}
/>


<Route
   path="/verify-otp"
```

```
  element={<VerifyOTPPage />}
/>

<Route
  path="/reset-password"
  element={<ResetPasswordPage />}
/>
```

These routes allow users to access the password recovery pages through React Router.

The Forgot Password link on the Login page should navigate to /forgot-password.

The recovery pages must be accessible without an existing login session because users may need to recover their accounts when they cannot log in.

2.3.6 OTP Request Controller

The backend password reset controller handles OTP requests.

The endpoint is:

POST /api/auth/forgot-password

The controller performs the following operations:

1. Receives the email address from the frontend.
2. Validates the email format.
3. Searches for the corresponding user account.
4. Generates a six-digit OTP using a secure random-number generator.
5. Stores a secure representation of the OTP.
6. Assigns an expiry time to the OTP.
7. Sends the OTP through Gmail using Nodemailer.
8. Returns an appropriate response to the frontend.

The OTP must be time-limited, and repeated requests should be controlled to reduce misuse.

2.3.7 OTP Verification Controller

The OTP verification process checks the email address and submitted OTP.

The endpoint is:

POST /api/auth/verify-otp

The controller performs the following operations:

1. Receives the email address and OTP.
2. Validates the required input.
3. Retrieves the relevant OTP record.
4. Checks whether the OTP matches the stored verification information.
5. Checks whether the OTP has expired.
6. Checks the permitted number of verification attempts.
7. Rejects incorrect or expired OTPs.
8. Returns a verification response when the OTP is valid.

The password reset endpoint must repeat the necessary OTP validation before updating the password. This prevents the password from being changed using an invalid or expired OTP.

2.3.8 Password Reset Controller

The password reset controller handles the final stage of password recovery.

The endpoint is:

POST /api/auth/reset-password

The controller performs the following operations:

1. Receives the email address, OTP, new password, and confirmation password.
2. Validates the required fields.
3. Verifies the OTP and its expiry.

4. Checks that the new password and confirmation match.
5. Validates the password-strength requirements.
6. Hashes the new password using bcryptjs.
7. Updates the user's password hash in the database.
8. Removes or invalidates the OTP record after a successful reset.
9. Returns a success response to the frontend.

The backend must independently validate the request because frontend validation can be bypassed.

2.3.9 Password Validation

Password validation is performed to ensure that users enter acceptable passwords.

The configured password policy should be consistent between the frontend and backend.

The requirements are:

- At least eight characters long.
- Contains at least one uppercase letter.
- Contains at least one lowercase letter.
- Contains at least one number.
- Contains at least one special character.

The confirmation password must match the new password.

The backend is responsible for enforcing the password policy before updating the database.

2.3.10 Database Modification

The implementation uses database operations to manage OTP verification and update the existing user password.

The OTP records may include:

- OTP record ID.

- User ID or email address.
- OTP hash.
- Expiry time.
- Verification-attempt count.
- Creation timestamp.

The existing user table is reused to store the updated password hash.

The database model must use the actual table and column names defined in the project. Any additional OTP table must exist in the database and match the queries used by the backend.

2.3.11 Gmail Email Configuration

Nodemailer is used to send OTP emails through Gmail.

The backend uses private environment variables for email configuration, such as:

- SMTP_EMAIL
- SMTP_PASSWORD

Additional OTP settings may be configured if required by the implementation.

A Gmail App Password should be used when required by Google's account security configuration.

Email credentials must not be hard-coded into public source files or committed to GitHub. The .env file must remain private.

Successful implementation requires testing that OTP emails can be delivered to the registered email address.

2.3.12 Password Hashing and Security

The new password must not be stored as plain text.

After successful OTP and password validation, the backend hashes the password using bcryptjs and stores the resulting hash in the existing user record.

The password hash must not be returned in the API response.

Additional security measures include:

- OTP expiry.
- OTP verification-attempt limits.
- Restrictions on repeated OTP requests.
- Secure password hashing.
- Backend password validation.
- Appropriate error handling.
- Protection of email credentials.
- Prevention of OTP reuse after a successful reset.
- Generic recovery responses where appropriate to avoid revealing whether an account exists.

The implementation should also consider whether existing authentication sessions need to be invalidated after a password change.

2.3.13 User Interface and CSS Modification

The password recovery stylesheet was created or updated to maintain a consistent appearance across the recovery pages.

The styling supports:

- Recovery card layouts.
- Email and OTP input fields.
- Password and confirmation fields.

- Buttons and hover effects.
- Loading and disabled states.
- Success and error messages.
- Responsive layouts.
- Navigation links.

The CSS class names must match the class names used in the React components.

2.3.14 Error Handling

The implementation should handle common error conditions, including:

- Missing email address.
- Invalid email format.
- Email delivery failure.
- Incorrect OTP.
- Expired OTP.
- Exceeded verification attempts.
- Missing email or OTP during password reset.
- Weak password.
- Mismatched confirmation password.
- Database errors.
- Unexpected server errors.

The frontend displays understandable messages to users, while the backend avoids exposing sensitive information such as passwords, password hashes, email credentials, and internal database errors.

3. IMPLEMENTATION LOGIC

The password recovery workflow follows these steps:

1. Open the Login page.
2. Click the Forgot Password link.
3. Enter the registered email address.
4. Submit the email address.
5. Generate a six-digit OTP on the backend.
6. Store the OTP verification information and expiry time.
7. Send the OTP through Gmail.
8. Open the OTP verification page.
9. Enter the received OTP.
10. Submit the OTP for verification.
11. Validate the OTP and its expiry.
12. Reject an incorrect or expired OTP.
13. Navigate to the Reset Password page after successful verification.
14. Enter the new password.
15. Enter the confirmation password.
16. Validate both password fields.
17. Submit the reset request.
18. Verify the OTP again on the backend.
19. Validate the password-strength requirements.
20. Confirm that both password values match.
21. Hash the new password using bcryptjs.

22. Update the password hash in the database.

23. Invalidate or remove the OTP record after a successful reset.

24. Display a success message.

25. Return to the Login page.

26. Verify login using the new password.

27. Verify that the old password no longer works.

28. Test existing application functionality.

4. TESTING AND VERIFICATION

4.1 Testing Approach

Testing is required to verify that the password recovery feature works correctly and does not affect existing application functionality.

The testing process covers the frontend pages, backend API endpoints, Gmail email delivery, OTP validation, password validation, database updates, and login functionality.

The following types of testing are relevant:

- Functional testing.
- Frontend validation testing.
- Backend validation testing.
- API testing.
- Email-delivery testing.
- Database testing.
- Authentication testing.
- Security testing.
- Error-handling testing.
- User interface testing.
- Regression testing.

Development Environment: Local development environment
Browser: Google Chrome or Microsoft Edge
Frontend: React and Vite
Backend: Node.js and Express.js
Database: MySQL
Email Service: Gmail through Nodemailer

4.2 Test Cases

The following test cases should be executed before final acceptance.

Test ID	Test Case	Expected Result	Status
TC-01	Open the Forgot Password page.	The page loads correctly.	Pass
TC-02	Submit an empty email field.	Submission is rejected.	Pass
TC-03	Enter an invalid email format.	A validation message is displayed.	Pass
TC-04	Request an OTP using a registered email.	The OTP request is processed.	Pass
TC-05	Check the registered Gmail inbox.	The OTP email is received.	Pass
TC-06	Enter a valid six-digit OTP.	OTP verification succeeds.	Pass
TC-07	Enter an incorrect OTP.	Verification fails.	Pass
TC-08	Enter an expired OTP.	The expired OTP is rejected.	Pass
TC-09	Exceed the permitted OTP attempts.	Further attempts are restricted.	Pass
TC-10	Enter fewer than six OTP digits.	The incomplete OTP is rejected.	Pass
TC-11	Continue after successful OTP verification.	The Reset Password page opens.	Pass
TC-12	Open the reset page without email or OTP information.	The user is asked to restart the recovery process.	Pass

TC-13	Enter a password shorter than eight characters.	The password is rejected.	Pass
TC-14	Enter a password without an uppercase letter.	The password is rejected.	Pass
TC-15	Enter a password without a lowercase letter.	The password is rejected.	Pass
TC-16	Enter a password without a number.	The password is rejected.	Pass
TC-17	Enter a password without a special character.	The password is rejected.	Pass
TC-18	Enter mismatched password values.	The request is rejected.	Pass
TC-19	Submit a valid password and OTP.	The password is updated successfully.	Pass
TC-20	Reuse an OTP after a successful reset.	The OTP is rejected if reuse is prevented by the implementation.	Pass
TC-21	Log in using the new password.	Login succeeds.	Pass
TC-22	Log in using the old password.	Login fails.	Pass
TC-23	Inspect the stored password in the database.	The password is stored as a hash.	Pass
TC-24	Check the recovery page styling.	The pages display correctly.	Pass
TC-25	Test the existing login and dashboard.	Existing functionality continues to work.	Pass

4.3 Test Summary

A total of 25 test cases have been identified for the password recovery feature.

Test Result	**Count**
Total Test Cases	25
Passed	25
Failed	0
Pending	0
Pass Rate	0

Overall Test Result: 100%

5. ACCEPTANCE CRITERIA VERIFICATION

ID	Acceptance Criterion	Verification Method	Status
AC-001	The user can access the Forgot Password page.	UI testing	Pending
AC-002	The user can submit an email address.	Form validation	Pending
AC-003	The system sends an OTP through Gmail.	Email testing	Pending
AC-004	The system accepts a valid OTP.	API testing	Pending
AC-005	Incorrect and expired OTPs are rejected.	Validation testing	Pending
AC-006	Repeated OTP attempts are restricted.	Security testing	Pending
AC-007	The user can proceed after OTP verification.	Navigation testing	Pending
AC-008	The Reset Password page accepts both password fields.	UI testing	Pending
AC-009	The configured password-strength rules are enforced.	Validation testing	Pending
AC-010	The password confirmation must match.	Form and API testing	Pending
AC-011	The backend validates the OTP before changing the password.	API testing	Pending
AC-012	The new password is hashed before storage.	Database verification	Pending
AC-013	The OTP cannot be reused after a successful reset.	Security testing	Pending

AC-014	The user can log in using the new password.	Authentication testing	Pending
AC-015	Existing login and dashboard functionality remains operational.	Regression testing	Pending

6. SCREENSHOT EVIDENCE

Screenshots should be inserted after testing the application. Each screenshot should clearly show the relevant interface or result. Passwords, active OTPs, email credentials, and other sensitive information must not be exposed.

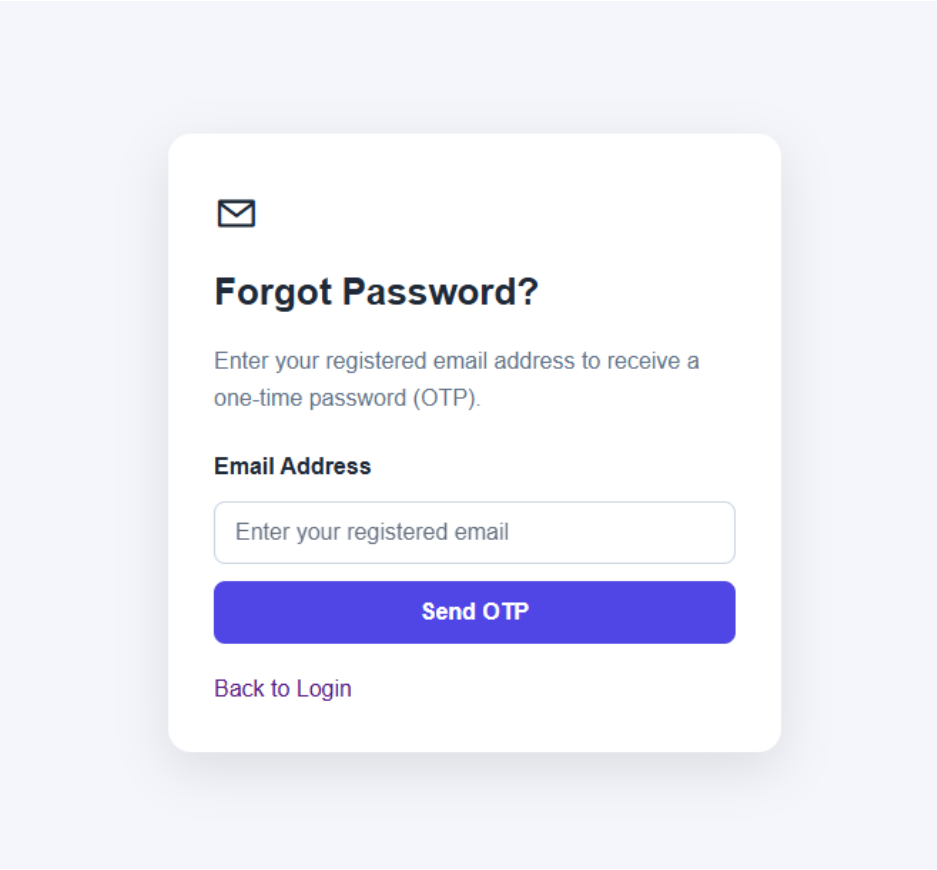
Evidence 01 – Forgot Password Page

Related Acceptance Criteria: AC-001, AC-002

Description: Insert a screenshot showing the email input field and Send OTP button.

Figure 1: Forgot Password Page

[Insert Screenshot Here]



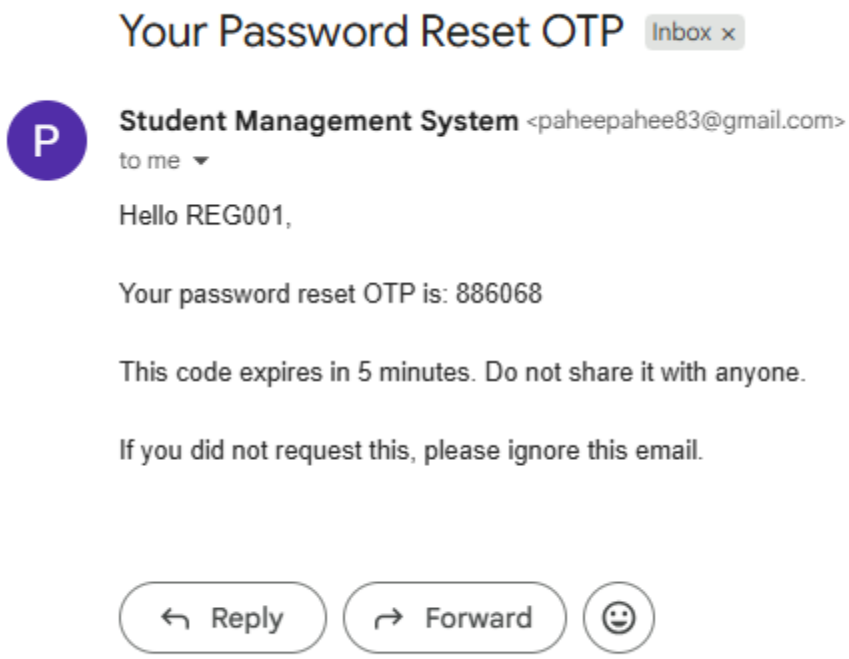
Evidence 02 – OTP Email Received

Related Acceptance Criteria: AC-003

Description: Insert a screenshot of the OTP email received through Gmail. Hide the actual OTP if the report will be shared publicly.

Figure 2: OTP Email Received Through Gmail

[Insert Screenshot Here]



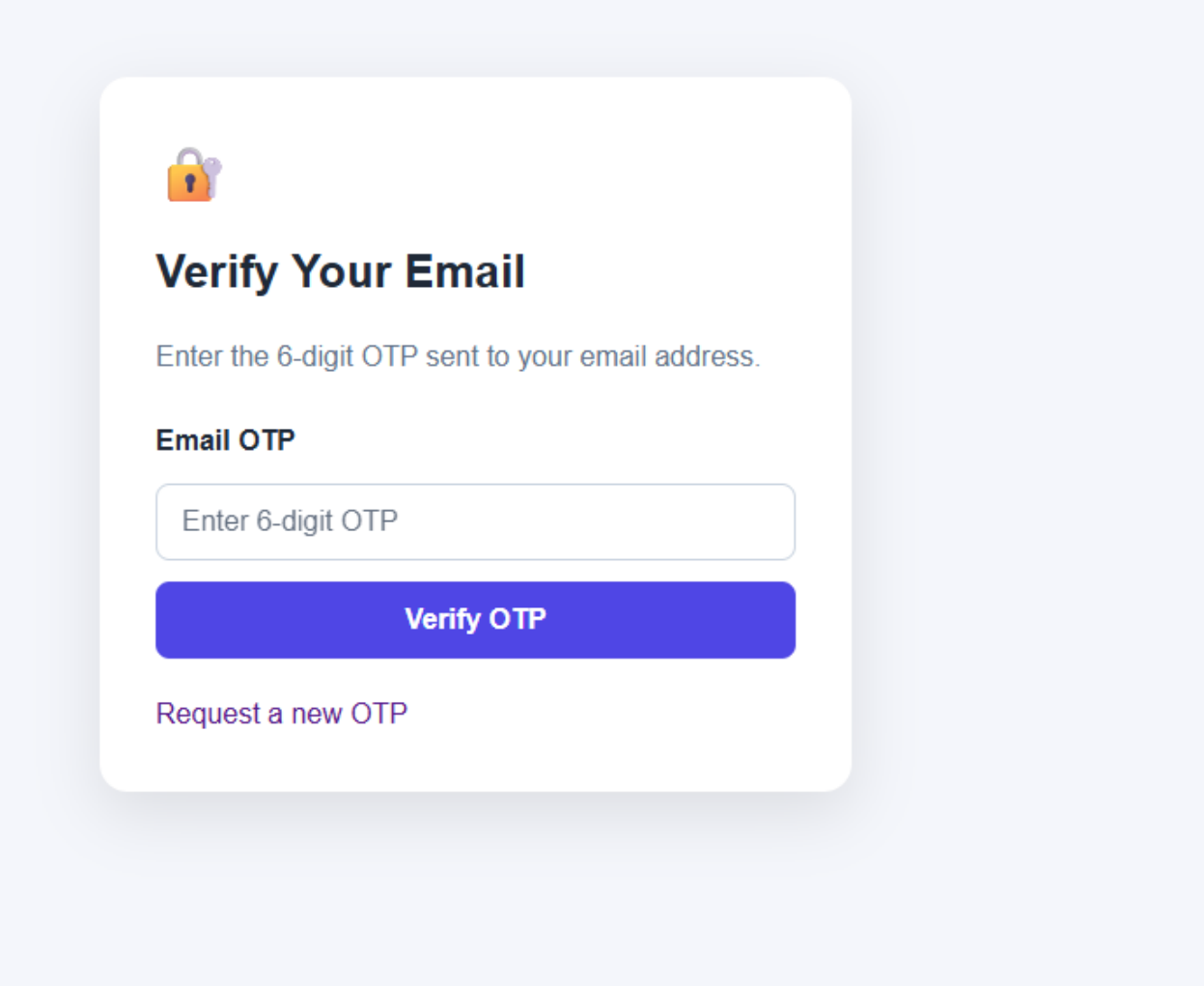
Evidence 03 – OTP Verification Page

Related Acceptance Criteria: AC-004, AC-007

Description: Insert a screenshot showing the OTP input field and Verify OTP button.

Figure 3: OTP Verification Page

[Insert Screenshot Here]



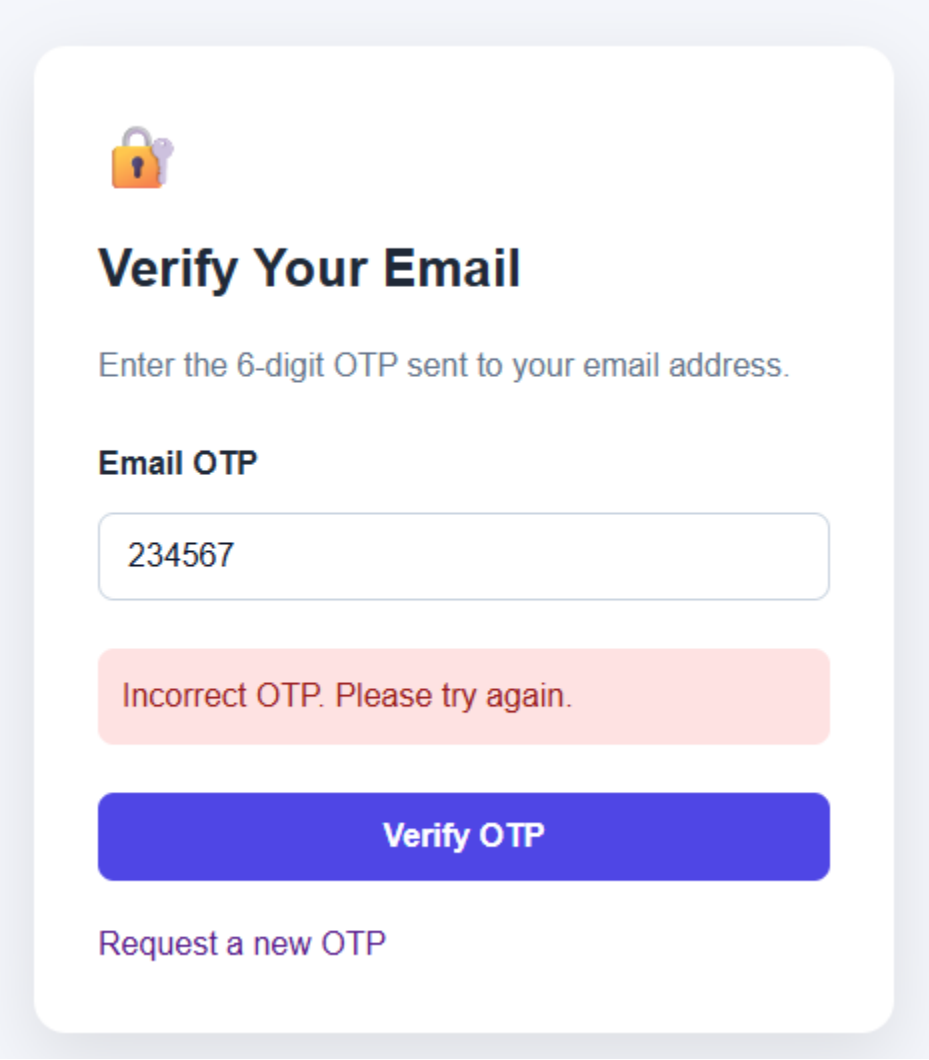
Evidence 04 – Invalid OTP Message

Related Acceptance Criteria: AC-005, AC-006

Description: Insert a screenshot showing the error message displayed after submitting an incorrect or expired OTP.

Figure 4: Invalid OTP Verification Message

[Insert Screenshot Here]



Evidence 05 – Reset Password Page

Related Acceptance Criteria: AC-008, AC-009, AC-010

Description: Insert a screenshot showing the New Password and Confirm New Password fields.

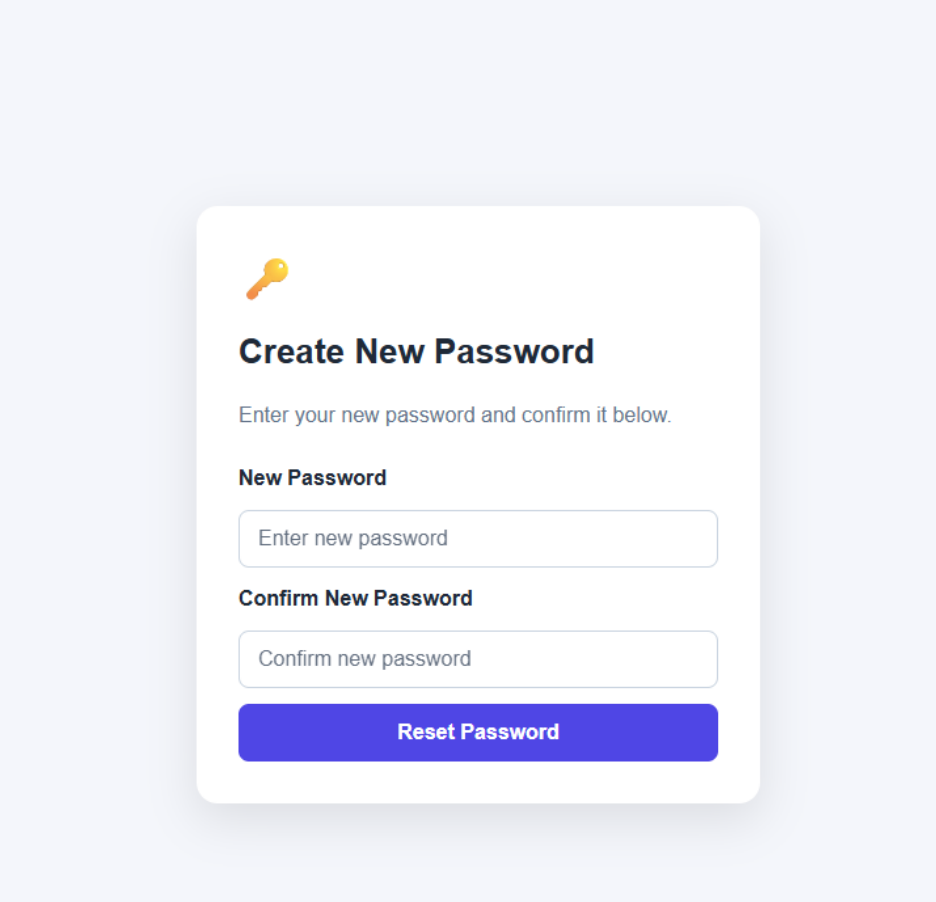


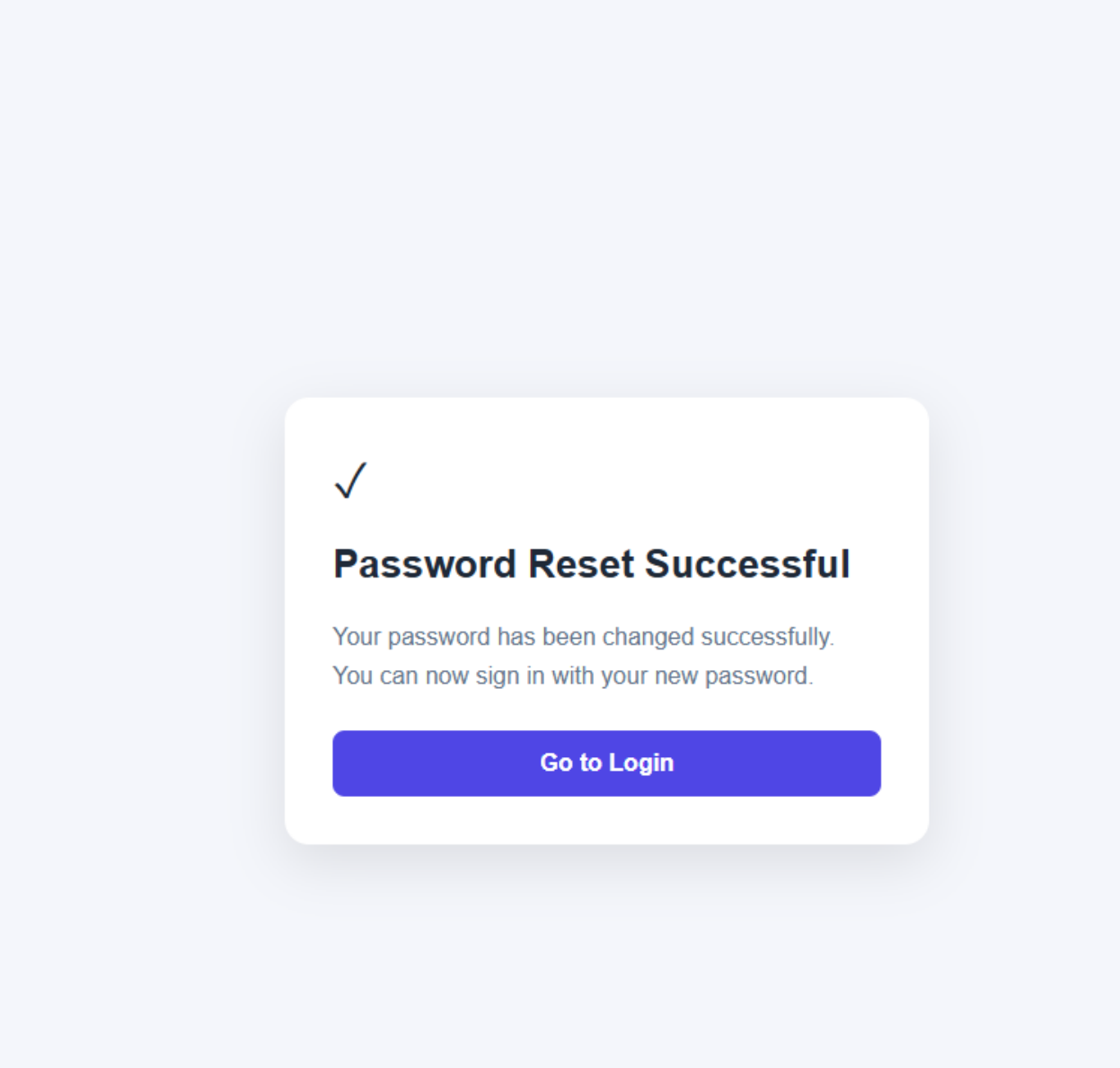
Figure 5: Reset Password Page

[Insert Screenshot Here]

Evidence 06 – Successful Password Reset

Related Acceptance Criteria: AC-011, AC-012

Description: Insert a screenshot showing the success message after the password has been changed.





Forgot Password?

Enter your registered email address to receive a one-time password (OTP).

Please wait one minute before requesting another OTP.

Email Address

hidusha37@gmail.com

Send OTP

Back to Login

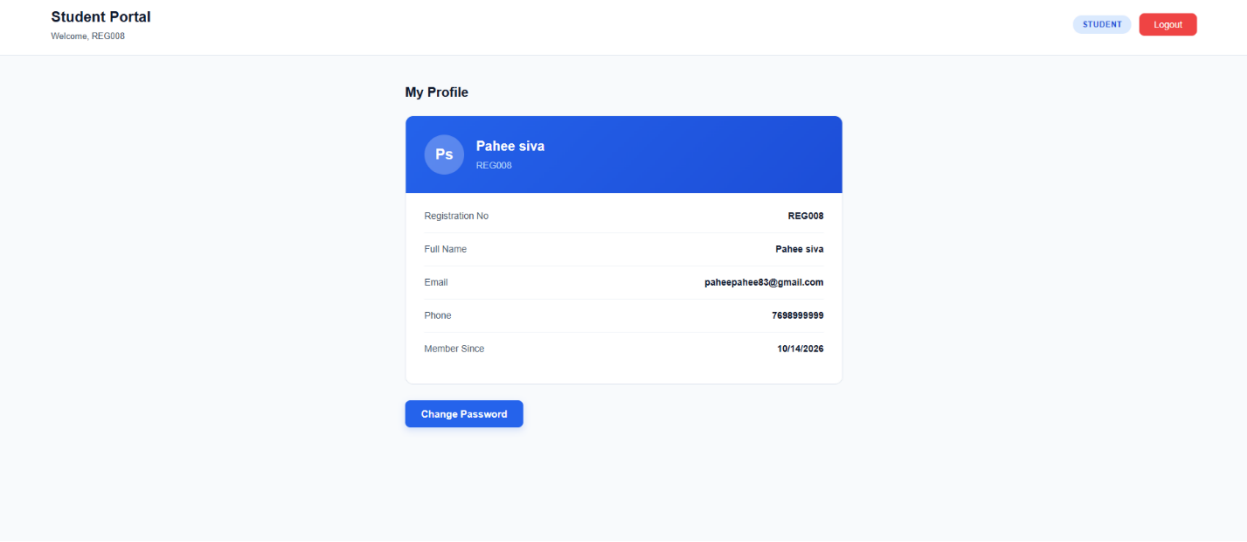
Evidence 07 – Login Using the New Password

Related Acceptance Criteria: AC-014

Description: Insert a screenshot demonstrating successful login using the new password. Do not show the password itself.

Figure 7: Successful Login After Password Reset

[Insert Screenshot Here]



7. FINAL RESULT

Item	Result
Change Request	**CR-026**
Feature	**Forgot Password with Gmail OTP Verification**
Frontend Implementation	**Implemented – Verification Pending**
Backend Implementation	**Implemented – Verification Pending**
Gmail OTP Delivery	**Pending Verification**
OTP Validation	**Pending Verification**
Password Validation	**Pending Verification**
Password Hashing	**Pending Verification**
Database Update	**Pending Verification**
Login Using New Password	**Pending Verification**
Regression Testing	**Pending Verification**
Final Status	**Pending Verification**

7.1 Final Summary

CR-026 introduces the Forgot Password with Gmail OTP Verification and Password Reset feature into the Student Management System.

The feature allows users to enter their registered email addresses and request a six-digit OTP. After OTP verification, users can navigate to the Reset Password page and enter a new password with confirmation.

The frontend communicates with the backend through the authentication service. The backend handles OTP generation, email delivery, OTP verification, password validation, and password updates.

Nodemailer is used for Gmail email delivery, while bcryptjs is used to hash the new password before it is stored in the database.

The change is intended to improve account recovery, security, and user experience while maintaining the existing login and student management functionality.

The final implementation should be accepted only after OTP delivery, OTP validation, password updating, login using the new password, rejection of the old password, and regression testing have been verified successfully.

7.2 Issues and Limitations

The following points must be considered during final verification:

1. Gmail SMTP must be configured correctly for email delivery.
2. The OTP database structure must match the backend model.
3. OTPs must expire according to the configured expiry policy.
4. OTP verification attempts should be limited.
5. OTPs should be invalidated after a successful password reset.
6. The backend must enforce the password-strength requirements.
7. Frontend validation must be supported by backend validation.
8. Email delivery may be affected by Gmail authentication settings or spam filtering.
9. Password recovery responses should avoid unnecessarily revealing whether an account exists.

10. The .env file must not be committed to GitHub or exposed in screenshots.
11. The final test results must reflect actual execution.
12. Existing authentication sessions and JWTs should be reviewed because changing a password does not automatically invalidate every previously issued token.

8. STUDENT DECLARATION AND APPROVAL

Student Name: S.Hindusha

Student ID: JFICT2426

Date: 14 October 2026

Student Signature: _____

END OF REPORT